Lottery Ticket Hypothesis

Rewinding y alcance de la poda en MLP, CNN, ResNet y ViT

Deep Learning y Sistemas Inteligentes - CC3092
Proyecto Final

José Antonio Mérida Castejón

Repositorio: `https://github.com/TonitoMC/lottery-ticket-hypothesis`
Video: `https://youtu.be/ow74cZd0Ldg`

2 de noviembre de 2026

Resumen

La *Lottery Ticket Hypothesis* (LTH) afirma que dentro de una red neuronal grande existe una subred pequeña que, entrenada desde sus mismos pesos iniciales, alcanza una accuracy comparable a la de la red completa. Este proyecto la reproduce y la pone a prueba en cuatro arquitecturas de complejidad creciente (MLP, CNN, ResNet de 20 capas y un ViT pequeño) con tres preguntas: (1) ¿existen *winning tickets* en cada arquitectura?, (2) ¿a medida que la red es más compleja hace falta hacer *rewind* de los pesos a un punto posterior del entrenamiento, y qué papel puede jugar el *warmup*?, y (3) ¿importa podar por capa o globalmente? Se encontraron tickets en las cuatro arquitecturas (en la ResNet solo con rewinding); la necesidad de hacer rewind es compatible con la complejidad de la red, pero está confundida con el *warmup* y el optimizador, por lo que no se puede atribuir a la arquitectura; y el tipo de poda cambió el resultado de la CNN, el caso donde se comparó directamente: con poda por capa el ticket colapsa con 10.7 % de los pesos (53.0 % frente a 61.2 % del control aleatorio) y con poda global no (75.7 % frente a 69.4 %), porque la poda por capa obliga a las capas pequeñas, cuyos pesos son los más grandes, a perder tanto como las grandes. Que las redes más grandes necesiten poda global es consistente con esto pero no se probó. Todos los experimentos son de una sola semilla.

Índice

1. Descripción del problema **3**
- 1.1. Contexto . . . 3
- 1.2. Problema . . . 3
- 1.3. Hipótesis . . . 3
- 1.4. Objetivos . . . 4

2. Análisis **5**
- 2.1. Iterative magnitude pruning (IMP) y definición de ticket . . . 5
- 2.2. Resultados esperados . . . 5
- 2.3. Alcance de la poda . . . 5
- 2.4. Rewinding y warmup . . . 6
- 2.5. Trabajo relacionado y límites de la comparación . . . 6

3. Propuesta de solución **7**

4. Descripción de la solución **8**
- 4.1. Arquitecturas . . . 8
- 4.2. Implementación . . . 10
- 4.3. Hiperparámetros . . . 10
- 4.4. Reproducibilidad y costo . . . 10

5. Herramientas aplicadas **11**
- 5.1. Herramientas . . . 11
- 5.2. Uso de IA generativa: prompts y evolución de la idea . . . 11

6. Resultados **14**
- 6.1. H0: existencia de tickets . . . 14
- 6.2. ViT con entrenamiento largo . . . 15
- 6.3. H1: rewinding según la arquitectura . . . 17
- 6.4. H2: ¿hace falta warmup? (observacional) . . . 17
- 6.5. H3: poda por capa frente a poda global (CNN) . . . 17
- 6.6. Inspección del ViT largo . . . 19

7. Discusión **24**
- 7.1. Discusión por hipótesis . . . 24
- 7.2. Limitaciones . . . 25

8. Conclusión **27**
- 8.1. Trabajo futuro . . . 27

A. Desviaciones respecto a las fuentes **29**

B. Cómo reproducir **29**

1. Descripción del problema

1.1. Contexto

Las redes neuronales modernas tienen millones de parámetros, pero la mayoría resulta prescindible. Luego del entrenamiento se pueden eliminar (*podar*) hasta el 90 % de los pesos de menor magnitud con poca pérdida de accuracy. Eso reduce memoria y cómputo en inferencia y tiene aplicación directa en la industria (modelos para dispositivos móviles y embebidos, menor costo energético) y en la academia (qué información realmente usa una red). Sin embargo, la poda tradicional se aplica a una red *ya entrenada* y no reduce el costo de entrenarla. Frankle y Carbin [1] mostraron que existe una alternativa, si la subred podada se reentrena *desde la misma inicialización* de la red original alcanza una accuracy igual o mayor. A esa subred la llaman *winning ticket*.

1.2. Problema

El resultado original se obtuvo en redes pequeñas (LeNet-300-100, Conv-2/4/6). En redes más profundas, regresar los pesos a la inicialización deja de funcionar con el *learning rate* estándar, y es necesario hacer rewind a una iteración temprana del entrenamiento [2]. Esto plantea la pregunta central del proyecto:

> *¿Cómo cambia la existencia de los tickets, y lo que hay que hacer para encontrarlos, al pasar de arquitecturas simples a arquitecturas más complejas (MLP, CNN, ResNet, ViT)? ¿Y qué decisiones del procedimiento de poda cambian ese resultado?*

1.3. Hipótesis

Se formulan cuatro hipótesis, de las que el proyecto prueba unas directamente y otras solo de forma observacional:

H0 – Existencia.
En cada arquitectura existe una subred con pocos pesos que, desde su inicialización (o desde un punto de rewind), iguala o supera a la red densa y supera a la *misma máscara con pesos nuevos* (control aleatorio). *Prueba directa:* las curvas de accuracy contra pesos restantes.

H1 – Complejidad y rewinding.
Mientras más compleja es la red, más tarde en el entrenamiento aparece el ticket: el MLP y la CNN no necesitan rewind, la ResNet sí, y el ViT también. *Prueba parcial:* se compara ticket desde θ_0 contra ticket con rewinding en cada arquitectura, pero con uno o dos puntos de rewind por arquitectura; no se barre el punto de rewind k.

H2 – Warmup.
El *warmup* del *learning rate* cumple una función parecida al rewinding (evitar el arranque inestable), por lo que una arquitectura con warmup necesita menos rewind. *Prueba observacional, no directa:* la ResNet se entrenó sin warmup y el ViT con warmup, pero el warmup no se manipuló de forma controlada. Está mezclado con el optimizador (SGD frente a AdamW), los datos y el tamaño de la red.

H3a – Alcance de la poda (CNN).
La poda *global* (un solo umbral de magnitud para toda la red) funciona mejor que la poda *por capa* (20 % de cada capa) porque no obliga a las capas pequeñas a perder la misma fracción que las grandes. *Prueba directa:* el experimento 02b repite el 02 cambiando únicamente el alcance de la poda.

H3b – Redes más grandes necesitan poda global.
A medida que la red crece (más capas, y de tamaños muy distintos), la poda por capa se vuelve más dañina porque las capas pequeñas se convierten en un cuello de botella. *No probada:* ResNet y ViT se podaron solo de forma global; la CNN es el único caso con ambas variantes.

1.4. Objetivos

1. Implementar poda iterativa por magnitud (IMP) con rewinding y control aleatorio, reutilizable en las cuatro arquitecturas.
2. Medir accuracy contra porcentaje de pesos restantes en cada arquitectura y para cada variante (ticket, ticket con rewinding, aleatorio).
3. Inspeccionar *qué* encontró la poda (pesos vivos por capa, máscaras, atención), no solo cuánta accuracy queda.
4. Documentar con honestidad qué se puede y qué no se puede concluir con los datos obtenidos.

2. Análisis

2.1. Iterative magnitude pruning (IMP) y definición de ticket

IMP repite el siguiente ciclo (Figura 1): (1) se entrena la red con la máscara actual; (2) se elimina una fracción p de los pesos vivos de menor magnitud $|w|$; (3) los pesos sobrevivientes se regresan a un valor de referencia; (4) se vuelve a entrenar. Con $p = 20\,\%$, tras n rondas quedan 0.8^n de los pesos ($10.7\,\%$ tras 10 rondas); con $p = 35\,\%$ en el ViT, 0.65^n.

El valor de referencia define el método: la inicialización θ_0 (ticket clásico [1]) o el valor tras k épocas de entrenamiento denso θ_k (rewinding [2]). Para saber si el resultado es mérito de la *estructura* o de los *valores iniciales*, se usa un control: la misma máscara con pesos reinicializados al azar. Si el ticket supera al control, los valores iniciales importan.

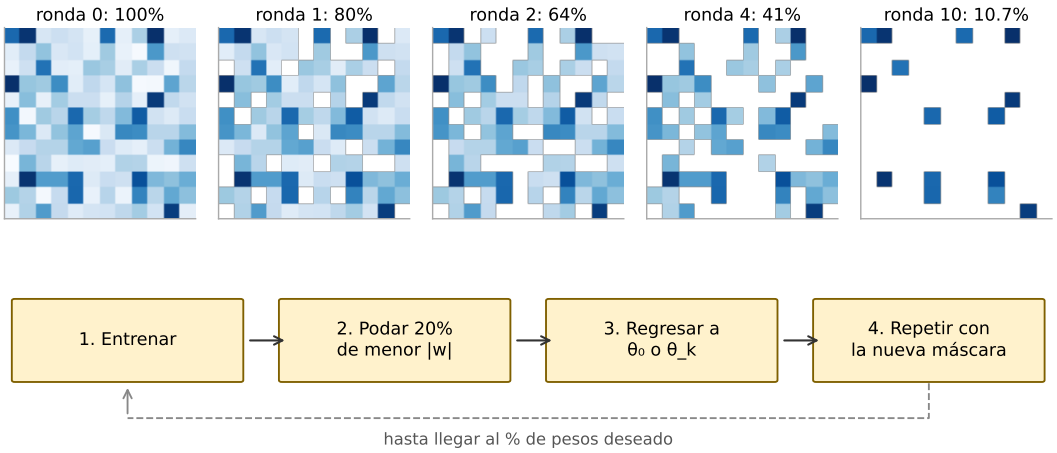


Figura 1: Poda por magnitud iterativa (IMP). Arriba, los pesos de una capa ilustrativa (12×12 valores aleatorios; más oscuro = mayor $|w|$, blanco = podado) tras 0, 1, 2, 4 y 10 rondas: con $p = 20\,\%$ quedan 0.8^n de los pesos. Abajo, el ciclo. En el paso 3 el valor de referencia define la variante: θ_0 (ticket), θ_k (rewinding) o pesos nuevos al azar (control). Es un esquema, no una corrida real.

2.2. Resultados esperados

- **Existencia (H0):** la curva del ticket se mantiene a menos de 1 punto de la red densa o por encima, y por encima de la curva aleatoria con un margen mayor al ruido.
- **Rewinding (H1):** si el ticket desde θ_0 no supera al control pero el ticket con rewind sí, hace falta rewind.
- **Alcance de la poda (H3):** si la caída del ticket a poca densidad desaparece al cambiar de poda por capa a poda global con todo lo demás igual.

2.3. Alcance de la poda

En la poda *por capa* cada capa pierde exactamente la misma fracción. Las capas pequeñas (como la primera convolución, con 1,728 pesos) quedan con muy pocos pesos en términos absolutos, mientras que las capas grandes (la primera capa lineal de la CNN tiene 2.1 M pesos, el 86 % del total) pierden en proporción lo mismo aunque sean redundantes. La poda *global* compara las magnitudes de todos los pesos con un único umbral, de modo que cada capa conserva lo que su magnitud justifica. El

riesgo opuesto es que una capa se quede sin pesos (*layer collapse*); aquí se mide con filtros y neuronas completamente vacíos.

2.4. Rewinding y warmup

Frankle et al. [2] muestran que en ResNet-18 y VGG-19 con *learning rate* 0.1 el ticket desde θ_0 no es mejor que la reinicialización, pero hacer rewind a una iteración temprana (cientos de iteraciones) sí lo es. Frankle y Carbin [1] (apéndice) reportan que en esas redes aparecen tickets con un *learning rate* menor o con warmup largo. Esto sugiere que el rewinding y el warmup atacan el mismo problema: el arranque del entrenamiento con un *learning rate* alto es inestable y los pesos iniciales aún no están en una región donde la máscara sea estable. Esta es la base de H2; en este proyecto no se manipuló el warmup, solo se observó su coincidencia con la necesidad de rewinding en cada arquitectura (Sección 6.4).

2.5. Trabajo relacionado y límites de la comparación

Para los ViT, Shen et al. [3] reportan que los tickets a nivel de peso son difíciles de encontrar en DeiT-Small con ImageNet; el ViT de este proyecto es pequeño y se entrena en CIFAR-10, así que sus resultados no son comparables directamente. Renda et al. [4] comparan el rewinding con el ajuste fino tras la poda; aquí solo se toma su receta (SGD con *weight decay* en ResNets de CIFAR-10) como referencia. El recipe de entrenamiento del ViT sigue las pautas de DeiT [5] (AdamW, *weight decay* 0.05, warmup), con las desviaciones del Apéndice A.

3. Propuesta de solución

Para contestar las preguntas se diseñó una serie de experimentos que comparten el mismo código de poda (`lth.py`) y varían la arquitectura, el punto de rewind y el alcance de la poda. Cada experimento mide las tres curvas de la Sección 2.1 (ticket, ticket con rewinding, control aleatorio) contra el porcentaje de pesos restantes.

Exp.	Arquitectura / datos	Poda	Rewind	Hipótesis que prueba	Épocas/ronda
01	MLP LeNet-300-100 / MNIST	capa	–	H0	5
02	CNN Conv-4 / CIFAR-10	capa	–	H0, H3	30
02b	CNN Conv-4 / CIFAR-10	**global**	–	**H3**	30
03	ResNet de 20 capas / CIFAR-10	global	época 1	H0, H1, (H2)	77
04	ViT pequeño / CIFAR-10	global	época 2	H0, H1, (H2)	20
05	ViT, entrenamiento largo	global	época 5	H0, H1, (H2)	100

Tabla 1: Matriz de experimentos. (H2) indica que el warmup solo se observa, no se manipula.

Enfoque del estudio. El proyecto no estudia las cuatro arquitecturas con la misma profundidad. El **ViT** es la arquitectura a la que se dedicó más trabajo: dos corridas (20 y 100 épocas) con las tres curvas, validación de su baseline y la inspección completa de los checkpoints (pesos vivos por capa, accuracy por clase y atención). La **CNN** es el caso donde se comparó directamente la poda por capa y la global, con inspección de máscaras. El MLP y la ResNet sirven como puntos de referencia (una corrida cada uno, la ResNet con rewinding).

Cambios deliberados entre experimentos. 02b cambia *solo* el alcance de la poda respecto a 02. 05 repite 04 con 100 épocas, más aumento de datos y *label smoothing* porque el baseline de 04 (78.9 %) parecía subentrenado y eso podría explicar que el ticket superara a la red densa. El ViT se mide dos veces (04 y 05) para separar el efecto de la poda del de un mal baseline.

Criterios de calidad. Cada curva se compara con la accuracy de su red densa y con el control aleatorio; la inspección de las máscaras y de las redes sirve para explicar las diferencias, no para reemplazar la medición. Se declara explícitamente que una sola semilla y 10,000 imágenes de prueba permiten distinguir diferencias de unos 5 puntos, pero no de 1 punto o menos.

4. Descripción de la solución

4.1. Arquitecturas

Cada red tiene su diagrama (Figuras 2 a 5); en todos, el verde marca las capas cuyos pesos entran a la poda.

MLP (LeNet-300-100). $784 \to 300 \to 100 \to 10$ con ReLU; 266k pesos podables, 235k de ellos en la primera capa.

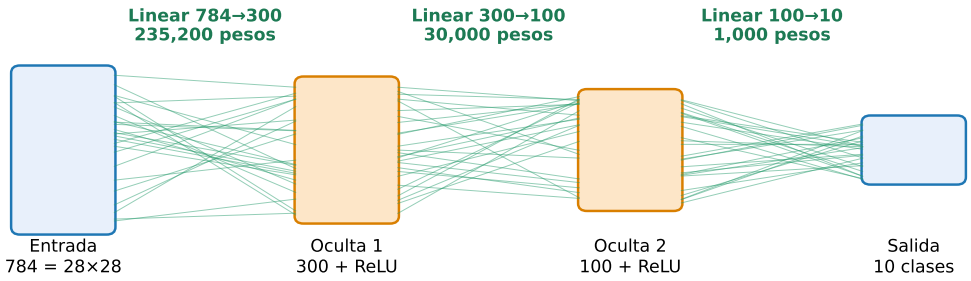


Figura 2: MLP LeNet-300-100 para MNIST. Cada barra es una capa de neuronas y las líneas verdes son los pesos que se podan.

CNN (Conv-4). Cuatro convoluciones 3×3 (64, 64, 128, 128 filtros) con *max-pooling* y tres capas lineales; 2.4M pesos, el 86 % en la primera capa lineal ($8192 \to 256$).

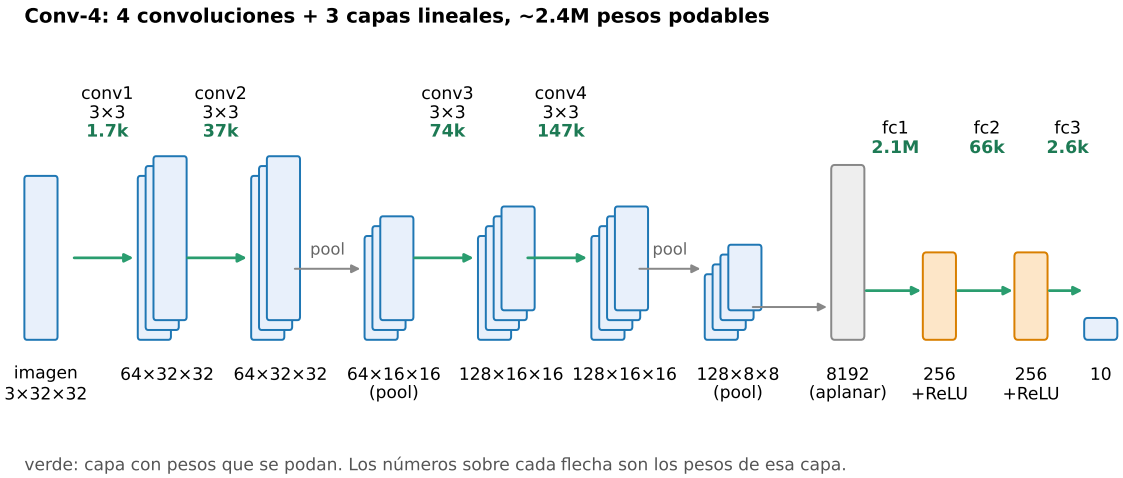


Figura 3: CNN Conv-4 para CIFAR-10: forma de cada tensor y pesos de cada capa. La primera capa lineal concentra el 86 % de los pesos.

ResNet de 20 capas. Arquitectura del apéndice I.3 de [1]: una convolución inicial y 9 bloques residuales en tres etapas de 16, 32 y 64 canales, con atajos convolucionales 1×1 cuando cambian las dimensiones; 270k pesos convolucionales. La capa lineal final no se poda.

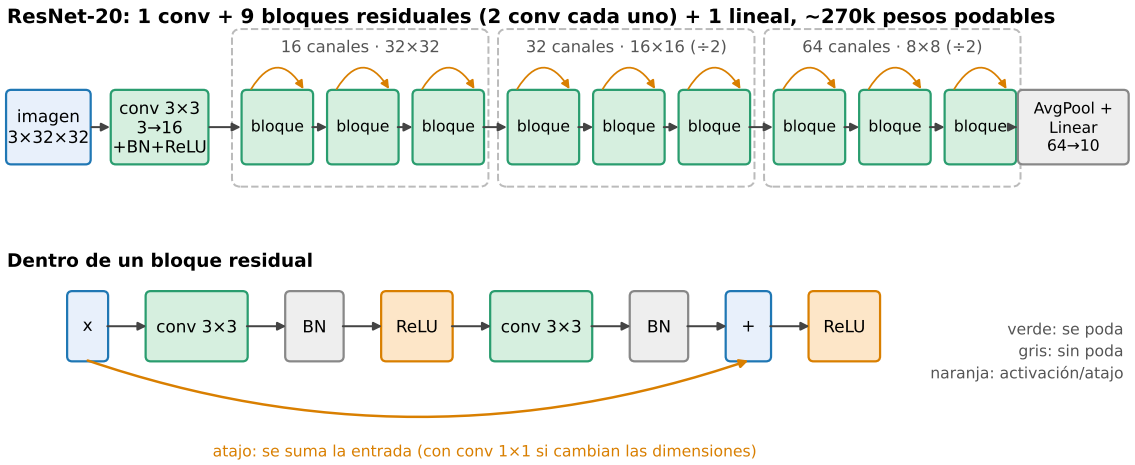


Figura 4: ResNet de 20 capas. Arriba, la red completa (los arcos son los atajos residuales); abajo, el interior de un bloque.

ViT pequeño. Parches de 4×4 (64 tokens más `[CLS]`), dimensión 192, 6 bloques de atención con 3 cabezas y MLP de 384; 1.8M pesos podables. Todas las capas lineales y las LayerNorm entran en la poda global.

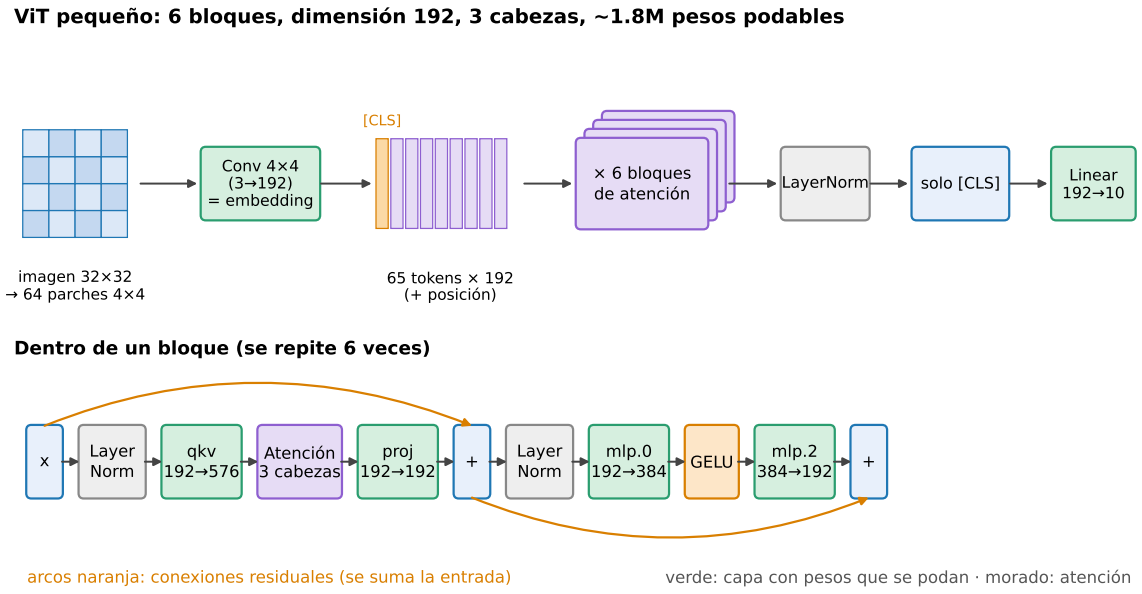


Figura 5: ViT pequeño. Arriba, de la imagen a la clasificación del token `[CLS]`; abajo, el interior de uno de los 6 bloques.

El ciclo de poda (IMP) que se aplica a todas estas redes está en la Figura 1.

4.2. Implementación

Los cuatro modelos, los ciclos de entrenamiento y evaluación y el módulo de poda `lth.py` fueron escritos por mí en PyTorch; el guardado de checkpoints y la mayor parte de los cuadernos de experimentos los implementó la IA (Sección 5.2). El módulo `lth.py` contiene:

- `run_imp`: bucle de IMP con alcance `"layer"` o `"global"`, ritmo de poda configurable, rewind a la época k y guardado de cada ronda;
- `run_lth`: ejecuta las tres curvas (ticket, ticket con rewinding, control aleatorio con la misma máscara y otra inicialización);
- `save_base`, `save_round` / `list_checkpoints` (implementados con IA): guardan pesos y máscaras de cada ronda para poder inspeccionarlos después sin reentrenar.

Los experimentos son notebooks `marimo` guardados como `.py` (reactivos y versionables), exportables a HTML. La comparación (06) y la inspección (07) leen los resultados guardados; la demo interactiva (08) usa los checkpoints para comparar modelos densos, tickets y controles sobre una imagen elegida.

4.3. Hiperparámetros

	MLP	CNN	ResNet	ViT (04)	ViT largo (05)
Optimizador	Adam 1.2e-3	Adam 3e-4	SGD 0.1, mom. 0.9	AdamW 1e-3, wd 0.05	igual
Aumento de datos	no	no	flip + crop	flip + crop	+ RandAugment, erasing, mixup/cutmix, LS
Poda por ronda	20 % capa	20 % capa / global	20 % global (solo conv.)	35 % global	35 % global
Rondas	15	10	10	8	8
Warmup	no	no	no	2 ép. + coseno	5 ép. + coseno

Tabla 2: Configuración de cada experimento (LS = *label smoothing*). Las desviaciones respecto a las fuentes están en el Apéndice A.

4.4. Reproducibilidad y costo

Entorno gestionado con `uv` (`uv.lock`); una sola semilla por experimento y entrenamiento en GPU (no determinista). Todo el proyecto usó unas 10 horas de GPU (RTX 4070 SUPER, 4 kWh). Las figuras y tablas de este informe se generan con `report/make_figures.py` a partir de `results/` y `checkpoints/`.

5. Herramientas aplicadas

5.1. Herramientas

- **PyTorch / torchvision**: modelos, entrenamiento y datasets (MNIST, CIFAR-10).
- **marimo**: cuadernos reactivos guardados como `.py`, exportables a HTML. Es una herramienta nueva para este curso (a diferencia de Jupyter, el orden de ejecución lo determina el grafo de dependencias entre celdas y el archivo es código normal, apto para git).
- **uv**: gestor de entornos reproducibles de Python.
- **Claude Code**: asistente de IA en la terminal, usado para leer el repositorio, ejecutar corridas, generar figuras de inspección y redactar borradores de análisis.
- **matplotlib**: figuras (el código de las figuras de inspección fue generado con ayuda de IA).
- **LaTeX**: este informe; las figuras y tablas numéricas se generan por script.

5.2. Uso de IA generativa: prompts y evolución de la idea

Nota sobre los prompts: no se conserva un registro textual completo de la conversación con la IA. Lo que sigue son resúmenes de las interacciones más relevantes, redactados a partir de lo ocurrido en el proyecto; las frases entre comillas son citas textuales (en inglés, como se escribieron).

Estrategia. Primero se escribió a mano una "implementación canónica" (modelos, ciclos de entrenamiento y evaluación, y `lth.py`) para el MLP y la CNN. Con ella como referencia, se le pidió a la IA conectar `lth.py` al resto de los cuadernos, ejecutar los experimentos y agregar los experimentos extra. Funcionó porque la IA tenía un patrón concreto que copiar (misma estructura de cuaderno, mismas funciones de poda, mismas métricas) y porque cada resultado se podía comparar contra la implementación a mano. Los problemas aparecieron cuando se le pidió algo sin esa referencia o cuando agregó cosas no pedidas.

Reparto.

Mío: el tema, la pregunta y las hipótesis; los cuatro modelos (`nn.Module`); los ciclos de entrenamiento y evaluación; el código de poda `lth.py` (máscaras, IMP, control aleatorio); y ejecutar y verificar los experimentos.

De la IA (Claude, mediante Claude Code): el guardado de pesos y máscaras (checkpoints); la implementación de la mayor parte de los cuadernos de experimentos, cuyas partes centrales yo sabía correctas y revisé; las visualizaciones, los diagramas de arquitectura y las tablas automáticas; la revisión de código e hiperparámetros contra las fuentes; borradores de interpretación; y la estructura del informe y de la presentación.

Estructura del informe y cómo se guió a la IA. El esqueleto de este informe (las secciones que pide el enunciado, el código LaTeX, los scripts que generan las figuras y tablas, y la estructura de la presentación) lo generó la IA a partir del enunciado y del contexto del proyecto; el tema, las hipótesis y el análisis son míos, y los borradores de texto que escribió la IA los revisé y los contrasté con los datos. Para que la IA se ciñera al experimento y no improvisara fui estricto con la forma: pedí que todos los cuadernos siguieran la misma estructura y las mismas métricas, que toda la lógica de poda viviera en `lth.py`, que el informe siguiera las secciones del enunciado y que lo no probado se redactara como tal (*compatible, no probada*) en lugar de presentarse como conclusión. Esa estructura obligó a la IA a trabajar sobre el diseño experimental que yo definí, y es la razón por la que la mayoría de los resultados se pudieron comparar contra mi implementación.

Prompts de los que hay registro.

- *"Modify the notebooks to export the models so I can run experiments on them, using masks to see how they turn out."* → origen del guardado de checkpoints (`save_round`) y del cuaderno de inspección.
- *"I wanna handpick from results to see what changes... also saving % of original params and whatnot."* → checkpoints por ronda con el porcentaje de pesos en el nombre.
- *"ViT longer has to have more epochs + more aggressive data augmentation (Luego de una conversación más larga sobre el experimento como tal)"* → experimento 05.

Tabla 3: Historial de prompts relevantes (resumido; lo que va entre comillas es cita, traducida cuando se indica).

Prompt (resumen)	Resultado	Evaluación y por qué
Reescribir los cuadernos 01 y 02 con mejor markdown y documentación, manteniendo la estructura.	Una versión con opciones no pedidas (una variable de entorno para correr en modo rápido, celdas extra) y que ignoraba mi código.	**No satisfactorio.** La IA agregó complejidad y no siguió las convenciones de `marimo` (por ejemplo, `print(model)` en vez de dejar el modelo como última expresión de la celda). Se evaluó comparando contra mi código; se pidió rehacerlo partiendo de él, solo con markdown y partes comunes extraídas.
Mover la lógica de poda a `lth.py`; los cuadernos solo cargan datos, definen el modelo, entrenan la versión inicial y llaman a `lth.py`.	Primero las máscaras se pasaban como parámetros y se usaban *hooks* del optimizador; las funciones no se parecían a las mías.	**Falló al inicio, corregido después de insistir.** "Los cuadernos quedan: cargar datos, modelo, entrenar versión inicial y guardar pesos; lo de poda y LTH va al archivo compartido" (traducido del inglés). Se validó revisando que las funciones fueran las mismas que ya tenía y que los cuadernos 01 y 02 quedaran con estructura equivalente.
Agregar el guardado de checkpoints por ronda con el porcentaje de pesos en el nombre.	`save_base`, `save_round`, `list_checkpoints` y el cuaderno de inspección.	**Satisfactorio.** Es una tarea acotada y fácil de verificar: se cargó un checkpoint y se comprobó que la máscara y el porcentaje coincidieran con lo reportado en las corridas.
Correr el ViT con más épocas y más aumento de datos para validar el *baseline* (experimento 05).	Cuaderno 05 con RandAugment, *erasing*, mixup/cutmix y *label smoothing*; la primera corrida del ViT denso se descartó y repitió.	**Satisfactorio con una corrección.** El cuaderno funcionó, pero la IA no había aplicado al 05 el arreglo de checkpoints que se le había pedido y hubo que repetir la instrucción. El resultado final se verificó contra el JSON de la corrida.
Quitar los `print` del conteo de parámetros en los cuadernos.	La IA los volvió a agregar al tocar otro cuaderno.	**Error.** Revirtió una decisión deliberada. No se sabe por qué; se supone que la IA no tenía presente que esos `print` se habían quitado a propósito. Se detectó al ver el resultado y se pidió no volver a agregarlos.
Correr la CNN con poda global (02b) para ver si el colapso se debe a la poda por capa.	Cuaderno 02b y los análisis de máscaras y de magnitud de pesos.	**Satisfactorio.** Antes de correrlo, el texto atribuía el colapso a la poda por capa como hecho; se reescribió como *posible, no probada* hasta tener 02b. Después se verificó con las máscaras (0 filtros vacíos) y con los pesos por capa.

Prompt (resumen)	Resultado	Evaluación y por qué
Figuras de atención, gradientes y oclusión del ViT, y borradores de interpretación.	Mapas de atención y texto que decía que el control aleatorio "mira abajo y por eso falla".	**Parcialmente erróneo.** La atención no es una explicación (Jain y Wallace, 2019); se midió y su correlación con la oclusión fue ≈ 0 (0.03 a 0.12). Se reescribió el texto, se marcó que las imágenes son elegidas a mano y se añadió la comparación atención contra oclusión.

Evaluación crítica de la IA. Todos los números que aparecen en el informe vienen de `results/*.json` o de los checkpoints; al crear la estructura del informe lo redactado con IA se marcó como *borrador* y se contrastó con los datos antes de aceptarlo. Además, se tomó en cuenta que la IA puede dar explicaciones causales con una gran certeza a pesar de que los datos no lo apoyen. Por ejemplo, en algunos puntos había concluido una relación causal cuando el experimento simplemente fue observacional o utilizó los mapas de atención como explicación.

6. Resultados

6.1. H0: existencia de tickets

La Figura 6 muestra las curvas de todas las corridas y la Tabla 4 las resume. Las tres curvas son: ticket desde θ_0 (azul), ticket con rewinding (verde) y reinicialización aleatoria de la misma máscara (naranja punteada).

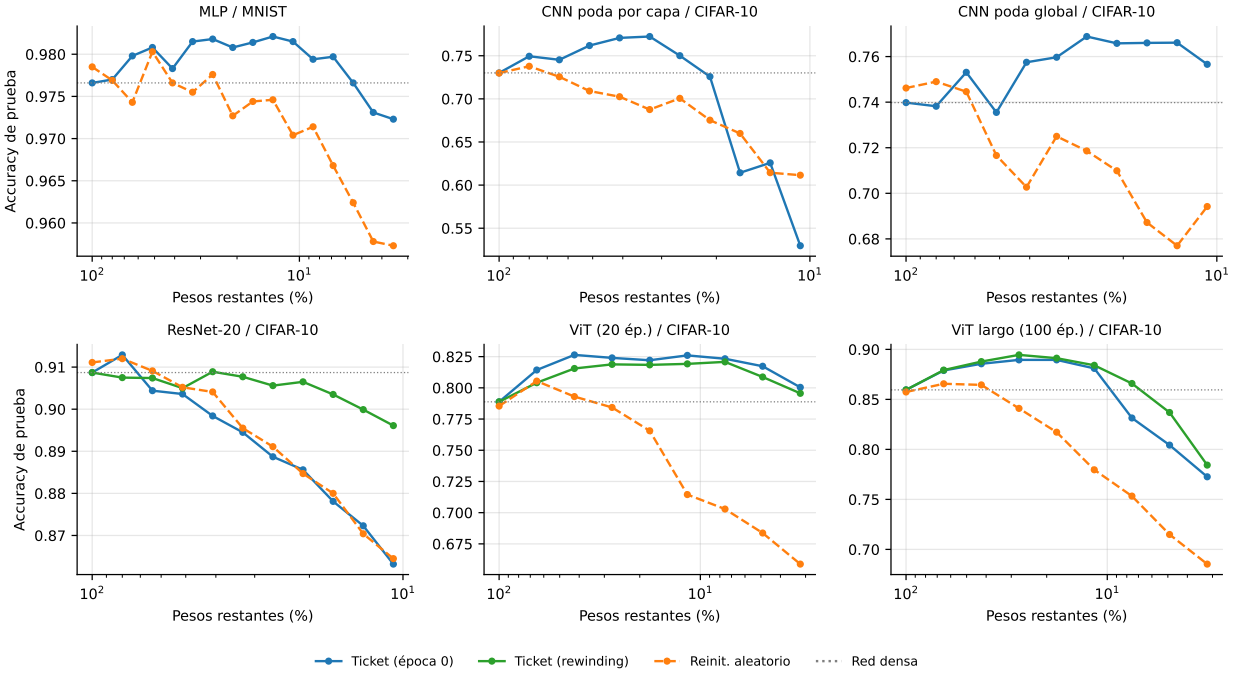


Figura 6: Accuracy de prueba contra porcentaje de pesos restantes (eje x logarítmico e invertido: la red se hace más pequeña hacia la derecha).

Arquitectura	Densa (%)	Mín. pesos ticket	Mín. pesos aleat.	Ventaja ticket (pts)	Ganancia rewind (pts)
MLP / MNIST	97.7	3.5 %	6.9 %	+0.8	–
CNN poda por capa / CIFAR-10	73.0	21.0 %	64.0 %	+2.2	–
CNN poda global / CIFAR-10	74.0	10.7 %	64.0 %	+4.4	–
ResNet-20 / CIFAR-10	90.9	51.2 %	41.0 %	-0.1	+1.5
ViT (20 ép.) / CIFAR-10	78.9	3.2 %	27.5 %	+8.1	-0.7
ViT largo (100 ép.) / CIFAR-10	86.0	11.6 %	42.3 %	+6.4	+1.1

Tabla 4: Resumen. “Mín. pesos”: la red más pequeña a menos de 1 punto de la accuracy densa. “Ventaja del ticket”: diferencia promedio entre el ticket y el control aleatorio sobre las rondas podadas. “Ganancia del rewind”: diferencia promedio entre el ticket con rewinding y el clásico.

MLP / MNIST. El ticket se mantiene entre 97.2 % y 98.2 % hasta 3.5 % de los pesos, a la par de la red densa (97.7 %), mientras que la reinicialización aleatoria cae a 95.7 %. No se probó el rewinding. En la primera capa, 128 de los 784 píxeles de entrada quedan sin ningún peso (el borde de la imagen), lo cual es esperable porque los píxeles que siempre valen 0 no reciben gradiente.

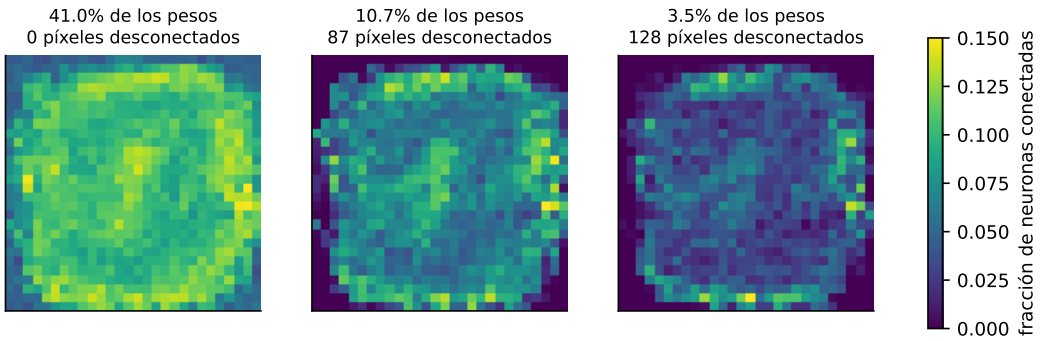


Figura 7: MLP: para cada píxel de la imagen de 28×28, la fracción de las 300 neuronas ocultas que sigue conectada a él en la primera capa, en tres niveles de poda.

Con 41 % de los pesos todavía todos los píxeles tienen alguna conexión; con 3.5 %, 128 píxeles del borde (donde los dígitos de MNIST casi siempre valen 0) quedan completamente desconectados y el resto se concentra en el centro de la imagen.

CNN / CIFAR-10. Con poda por capa el ticket supera a la red densa (73.0 %) hasta con 26 % de los pesos, con un máximo de 77.2 % con 33–41 %, y supera al control hasta con 21 %; por debajo de eso cae a 61.4 % con 16.8 %, 62.6 % con 13.4 % y 53.0 % con 10.7 %, por debajo del control en 16.8 % y 10.7 % (ver Sección 6.5). Con poda global el ticket se mantiene entre 73.6 % y 76.9 %, a la par o por encima de la red densa (74.0 %) en todo el rango y por encima del control desde 64 % de los pesos.

ResNet / CIFAR-10. Con *learning rate* 0.1 el ticket desde θ_0 **no** supera al control (86.3 % frente a 86.5 % con 10.7 % de los pesos), igual que en [2]. Hacer rewind a la época 1 recupera casi toda la accuracy: 89.6 % con 10.7 % de los pesos (red densa: 90.9 %).

ViT / CIFAR-10. Con 20 épocas el ticket queda en 82–83 % de 42 % a 7.5 % de los pesos, por encima de la red densa (78.9 %); el control cae a 65.9 % con 3.2 %. El baseline probablemente está subentrenado, motivo del experimento 05.

6.2. ViT con entrenamiento largo

El primer ViT (experimento 04, 20 épocas) quedó subentrenado: su red densa llegó a solo 78.9 %, y el ticket la superaba por varios puntos, lo que hacía sospechar que el resultado venía de un baseline débil y no de un ticket realmente bueno. Por eso se repitió con un entrenamiento largo (experimento 05: 100 épocas, warmup de 5 épocas y aumento de datos más fuerte). **Ese es el experimento principal del ViT**: es el que se usa para el resto del informe (rewinding, inspección de máscaras, atención), y el 04 queda como corrida preliminar.

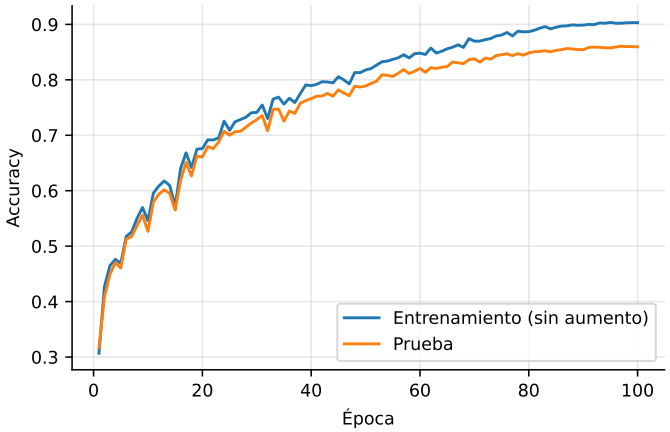


Figura 8: ViT denso de 100 épocas: accuracy de entrenamiento (sin aumento de datos) y de prueba por época.

Con 100 épocas la red densa llega a 86.0 % de accuracy de prueba (78.9 % en 04), con una brecha de unos 4 puntos respecto al entrenamiento y la prueba todavía subiendo: un baseline razonable. Medido otra vez desde otro arranque (ronda 0 del control aleatorio) da 85.7 %, así que la inicialización explica unos 0.2 puntos.

% de pesos	Ticket θ_0	Ticket rewind (ép. 5)	Aleatorio
100.0	86.0	86.0	85.7
65.0	87.9	87.9	86.6
42.3	88.6	88.8	86.5
27.5	88.9	89.5	84.1
17.9	88.9	89.1	81.7
11.6	88.1	88.4	78.0
7.5	83.1	86.6	75.3
4.9	80.4	83.7	71.5
3.2	77.3	78.4	68.5

Tabla 5: ViT largo (05): accuracy de prueba (%) por porcentaje de pesos restantes.

El ticket sigue por encima de la red densa hasta ∼12 % de los pesos, así que el resultado de 04 no era solo un baseline débil. El ticket le gana al control aleatorio en todos los puntos, y la diferencia crece al podar más (de unos 5 puntos con 27.5 % de los pesos a unos 9 con 3.2 %).

6.3. H1: rewinding según la arquitectura

Arquitectura	Rewind probado	¿Hizo falta?	Evidencia
MLP	no	no	el ticket desde θ_0 iguala a la red densa hasta 3.5 % de los pesos
CNN	no	no	ídem (con poda global hasta 10.7 %)
ResNet	época 1	**sí**	desde θ_0 igual al aleatorio (86.3 vs. 86.5 %); con rewind 89.6 %
ViT 20 ép.	época 2	no	rewind queda $\approx$ ticket (-0.7 puntos en promedio)
ViT 100 ép.	época 5	solo con poca densidad	sin efecto con $\geq$12 % de pesos; +3.4 puntos con 7.5 % y +3.3 con 4.9 %, +1.2 con 3.2 %

Tabla 6: Necesidad de rewinding por arquitectura.

Los datos son compatibles con la idea de que a más complejidad se necesita hacer rewind (MLP y CNN no, ResNet sí), pero el ViT, la arquitectura más compleja, no lo necesita con densidades medias. La relación no es monótona en la complejidad; la explicación que más compite es H2 (Sección 6.4).

En el ViT largo el rewinding solo ayuda cuando quedan pocos pesos, con 12 % o más la subred aún tiene capacidad para recuperarse desde cualquier inicialización razonable, y ticket, rewind y red densa quedan dentro de $\sim$1 punto. Con 7.5 % y 4.9 % la subred tiene poca capacidad y su entrenamiento es más sensible; Frankle et al. [2] encuentran que el ticket solo es estable si se arranca después de unas pocas iteraciones. Las 5 épocas del rewinding coinciden con el final del warmup, lo que es compatible con esa idea pero no se probó. En 04 el baseline subentrenado dejaba poco margen, y no tiene los puntos 7.5 % y 4.9 % donde 05 muestra la ganancia; las dos corridas no son comparables punto por punto.

6.4. H2: ¿hace falta warmup? (observacional)

La ResNet (sin warmup, *learning rate* 0.1 desde la época 0) necesita rewinding; el ViT (con warmup de 2 y 5 épocas) funciona desde θ_0 y solo aprovecha el rewind con muy poca densidad. Como el warmup no se varió dentro de una misma arquitectura, y las dos difieren además en optimizador, tipo de capas, etc., no se puede concluir que el warmup haga el trabajo del rewinding. Sin embargo, lo observado sí es compatible con esa hipótesis. Como prueba directa a futuro, se repetiría la ResNet con warmup y el ViT sin warmup: se esperaría que el ticket desde θ_0 aparezca en la ResNet y desaparezca en el ViT.

6.5. H3: poda por capa frente a poda global (CNN)

Con poda por capa el ticket de la CNN cae a partir de 16.8 % de los pesos y llega al mínimo con 10.7 %: 49.7 % frente a 61.0 % del control en la primera corrida, y 53.0 % frente a 61.2 % en la repetición (resultados de este informe), por lo que no es ruido. Con poda global (02b) el colapso desaparece: 75.7 % para el ticket frente a 69.4 % del control (red densa: 74.0 %). Pasa de 8 puntos *por debajo* del control a 6 puntos *por encima*.

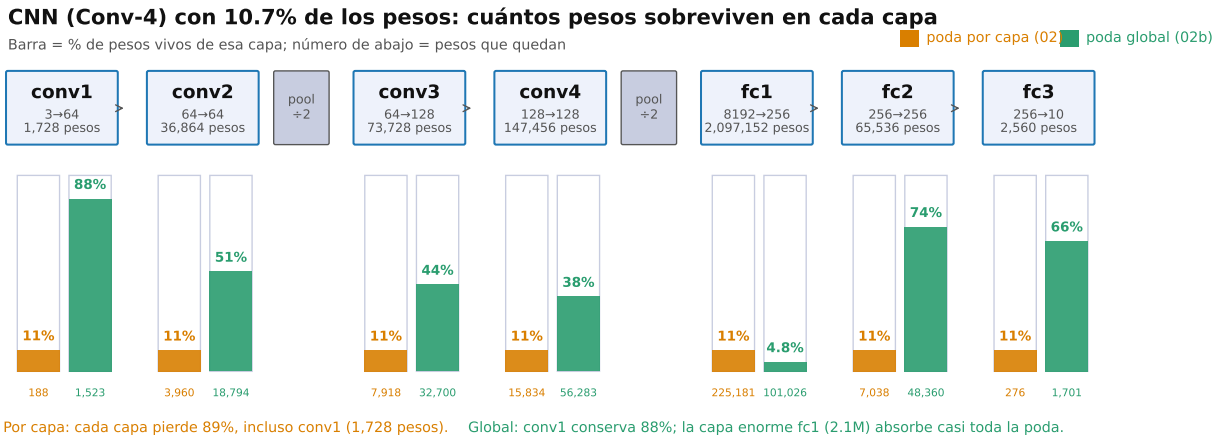


Figura 9: La CNN (Conv-4) con 10.7 % de los pesos totales. Sobre cada capa: forma y número de pesos; debajo: fracción de pesos vivos de esa capa con poda por capa (naranja) y poda global (verde), y número de pesos que quedan. Con poda por capa todas las capas quedan en ∼11 %; con poda global conv1 conserva 88 % y fc1 solo 4.8 %.

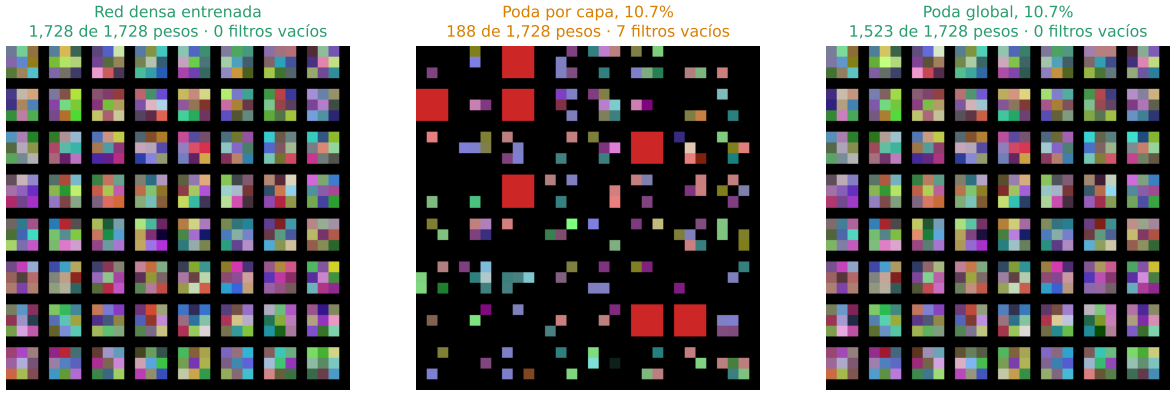


Figura 10: Los 64 filtros de la primera convolución de la red densa, del ticket con poda por capa y del ticket con poda global (10.7 % de los pesos). Cada cuadro es un filtro $3 \times 3 \times 3$ (el color son los tres canales). Negro: posición podada. Rojo: filtro sin ningún peso vivo.

	Por capa (02)			Global (02b)		
Capa	% vivo	Pesos	Vacíos	% vivo	Pesos	Vacíos
conv1 (3→64)	10.9	188	7/64	88.1	1 523	0/64
conv2 (64→64)	10.7	3 960	0/64	51.0	18 794	0/64
conv3 (64→128)	10.7	7 918	0/128	44.4	32 700	0/128
conv4 (128→128)	10.7	15 834	0/128	38.2	56 283	0/128
fc1 (8192→256)	10.7	225 181	–	4.8	101 026	–
fc2 (256→256)	10.7	7 038	–	73.8	48 360	–
fc3 (256→10)	10.8	276	102/256	66.4	1 701	0/256

Tabla 7: CNN, ronda 10 (10.7 % de los pesos). "Vacíos": filtros convolucionales sin ningún peso vivo; en fc3, neuronas ocultas sin ningún peso hacia la salida.

Con poda global la primera convolución conserva 88 % de sus pesos, no hay filtros ni neuronas vacíos (Tabla 7) y la primera capa lineal, que tiene 86 % de los pesos, queda en 4.8 %. Con poda por capa hay 7 filtros vacíos en conv1 y 102 de las 256 neuronas ocultas no llegan a la salida.

Limitaciones. Una sola corrida por configuración, con inicializaciones distintas en 02 y 02b y dense baselines de 73.0 % y 74.0 % (el ruido entre corridas es de ∼1 punto). Además, 02b cambia a la vez el alcance de la poda y cuántos pesos conserva cada capa; no separa ambas cosas.

6.6. Inspección del ViT largo

Checkpoints de 05 a 11.6 % de los pesos (ronda 5). `07_inspection` tiene vistas adicionales (cabezas de atención, IoU entre ticket y rewinding, forma de las máscaras).

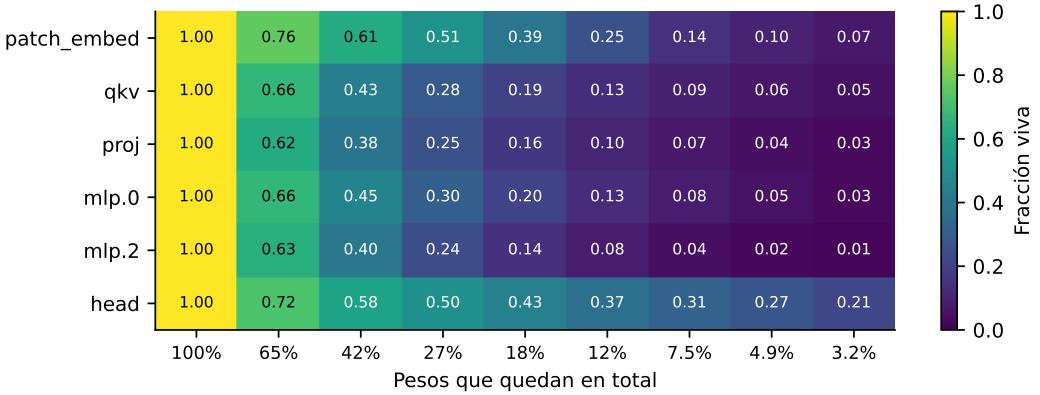


Figura 11: ViT largo: fracción viva por tipo de capa y ronda de poda (promedio ponderado sobre los 6 bloques).

Pesos vivos por capa. Clara = casi todos los pesos vivos. En la última ronda (3.2 % del total) `head` conserva 21 %, `patch_embed` 7 %, `qkv` 4.5 %, `mlp.0` 2.7 %, `proj` 2.6 % y `mlp.2` 1.3 %.

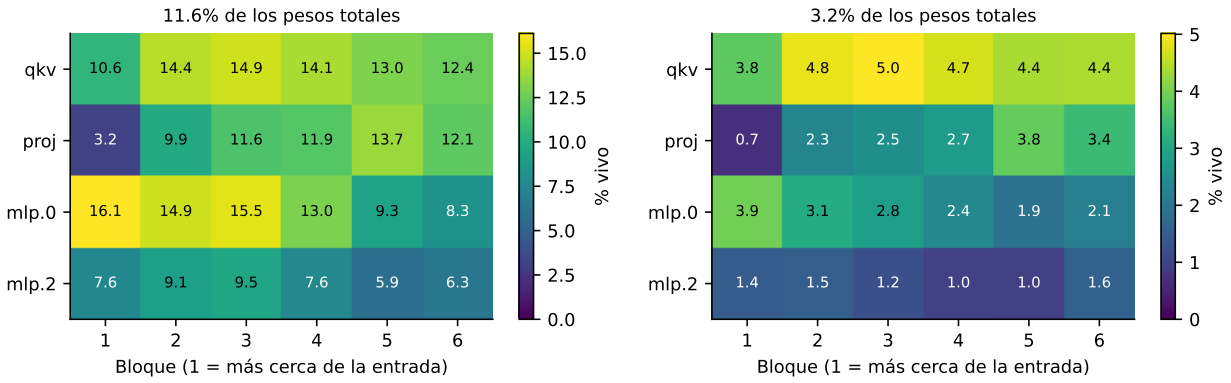


Figura 12: ViT largo: porcentaje de pesos vivos de cada capa dentro de cada bloque (1 = el más cercano a la entrada), con 11.6 % y 3.2 % de los pesos totales.

Cada celda es una capa de un bloque. `mlp.0` conserva menos pesos en los bloques tardíos (16.1 % en el bloque 1 y 8.3 % en el 6 con 11.6 %), `proj` lo contrario (3.2 % a 12.1 %) y `qkv` es casi plano. `proj` del bloque 1 queda en 0.7 % con 3.2 % de los pesos.

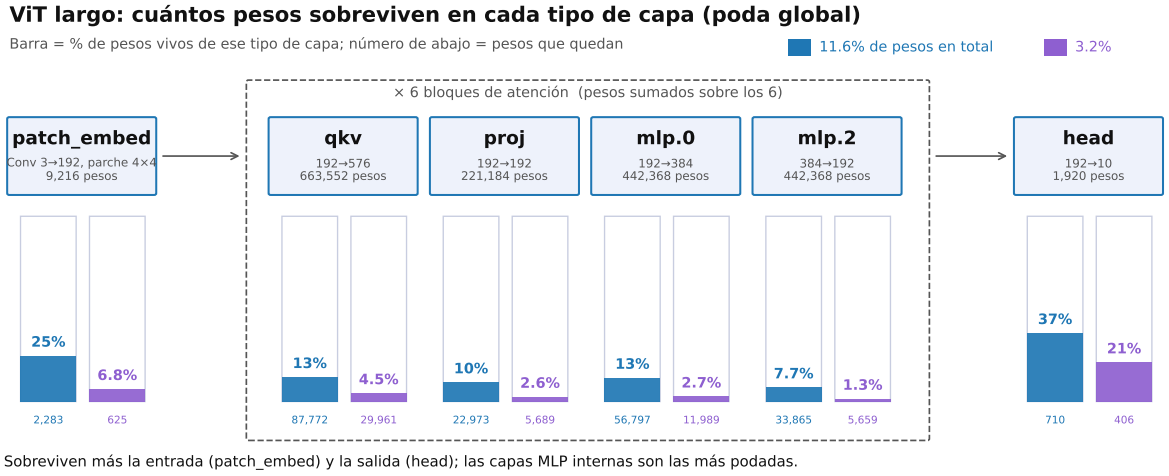


Figura 13: ViT largo: fracción de pesos vivos por tipo de capa (sumados sobre los 6 bloques) con 11.6 % y con 3.2 % de los pesos totales, y número de pesos que quedan.

Misma información que el mapa de calor, ordenada como la red. Las capas extremas conservan más y `mlp.2` es la más podada.

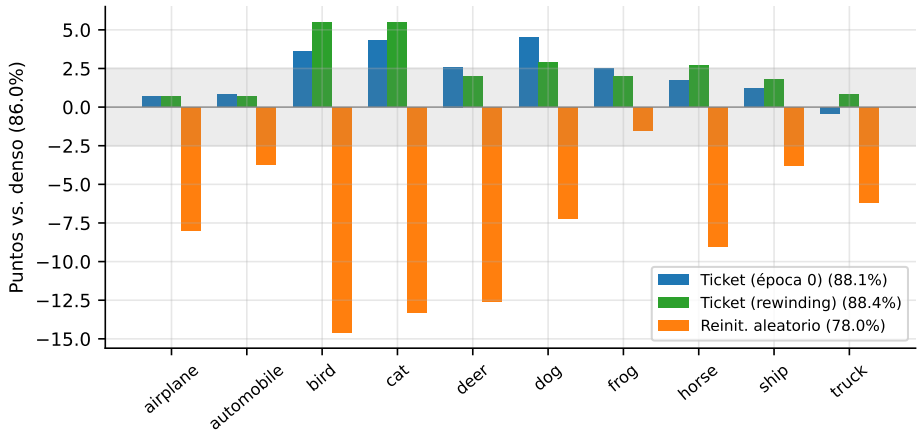


Figura 14: ViT largo: diferencia de accuracy por clase respecto al denso (11.6 % de los pesos). Banda gris: ruido de ±2.5 puntos.

Accuracy por clase. Positivo = acierta más que el denso en esa clase; solo cuenta lo que sale de la banda gris (1,000 imágenes por clase). El ticket queda entre −0.4 y +4.5 puntos del denso, el rewinding entre +0.7 y +5.5 y el control pierde en todas las clases (hasta 14.6 puntos en pájaro).

Atención y oclusión. Los mapas de atención del `[CLS]` (último bloque, promedio de cabezas) se suelen leer como "dónde mira el modelo", y la idea era usarlos para ver qué conserva el ticket y qué pierde el control aleatorio. Pero la atención no siempre refleja qué usa el modelo para decidir (Jain y Wallace, 2019), así que antes de interpretarla se comparó con una medida más directa, la oclusión: se tapa cada parche con el color medio de la imagen y se mide cuánto baja la probabilidad de la clase correcta. Se hizo con 150 imágenes de prueba y los cuatro modelos a 11.6 % de los pesos.

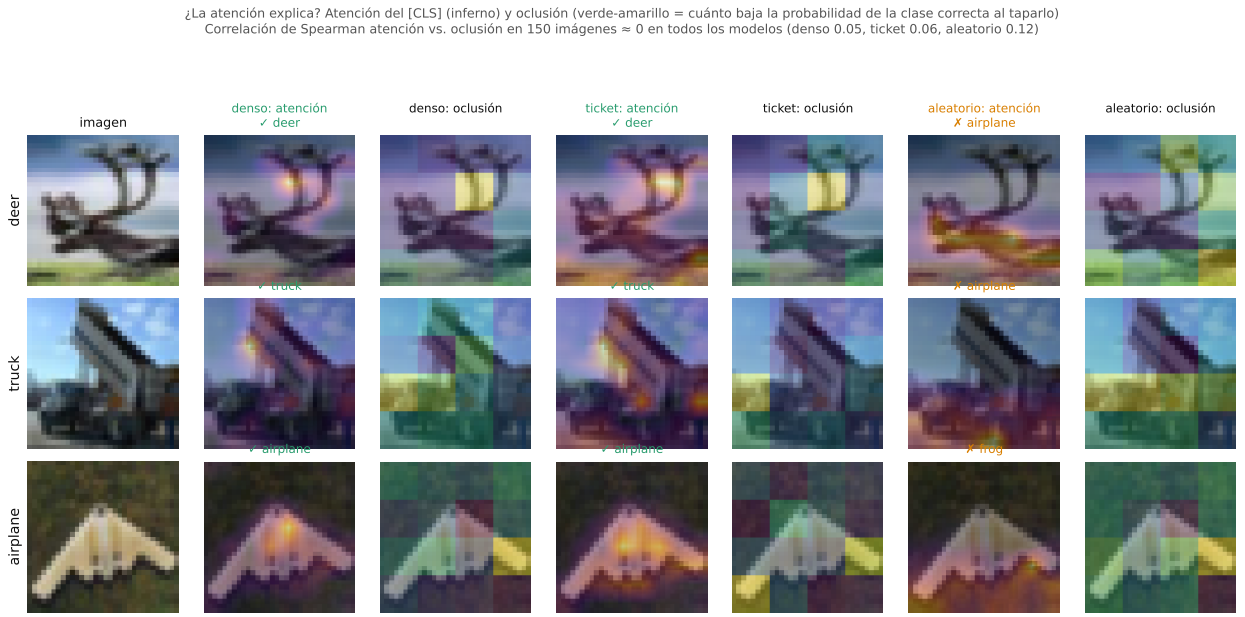


Figura 15: Atención del `[CLS]` y oclusión de bloques de 8×8 píxeles (amarillo = al taparlo baja más la probabilidad de la clase correcta) para el denso, el ticket y el control aleatorio, con 11.6 % de los pesos. **Las tres imágenes están elegidas**: son de las 140 (entre las primeras 1,500 de prueba) en las que denso, ticket y rewinding aciertan y el control falla; no son representativas.

Hallazgo 1: la atención no coincide con la oclusión. La correlación de Spearman entre ambas es ≈ 0 en los cuatro modelos: 0.05 (denso), 0.06 (ticket), 0.03 (rewinding) y 0.12 (aleatorio) con bloques de 8×8; con 4×4, 0.01, 0.03, 0.01 y 0.07; con *attention rollout*, 0.04 a 0.09. Los parches a los que más atiende el modelo no son los que más afectan la predicción al taparlos, así que los mapas de atención se presentan como descripción y no se usan para explicar nada.

Hallazgo 2: los mapas del control aleatorio son distintos. Los mapas del denso, el ticket y el rewinding se parecen entre sí (Tabla 8: 0.63 a 0.73 en atención y 0.48 a 0.55 en oclusión); los del control casi no se parecen a los demás en atención (0.17 a 0.28) y se parecen menos en oclusión (0.33 a 0.35). Es un resultado raro: el control comparte la máscara con el ticket y aun así cambia bastante, y como la atención no explica la decisión no se puede decir por qué. Solo se registra que es distinto. Una reserva: denso, ticket y rewinding comparten θ_0 y el control no, y no hay referencia de cuánto se parecen dos redes densas entrenadas por separado (cifras en `results/map_similarity.json`).

Par de modelos	Atención	Oclusión
denso – ticket	0.63	0.48
denso – rewinding	0.70	0.49
ticket – rewinding	0.73	0.55
denso – control aleatorio	0.17	0.35
ticket – control aleatorio	0.28	0.35
rewinding – control aleatorio	0.24	0.33

Tabla 8: Correlación de Spearman media entre los mapas de dos modelos (11.6 % de los pesos, 150 imágenes de prueba; 1 = mismo orden). Una sola corrida por modelo; la oclusión con color medio puede introducir artefactos.

7. Discusión

7.1. Discusión por hipótesis

H0, existencia de tickets: apoyada. En las cuatro arquitecturas hay una subred pequeña que iguala o supera a la red densa y le gana al control aleatorio: en el MLP hasta 3.5 % de los pesos (97.7 % la red densa), en la CNN con poda global hasta 10.7 % (75.7 % frente a 74.0 %) y en el ViT largo hasta ∼12 % (por encima del denso de 86.0 %). En la ResNet solo aparece con rewinding: 89.6 % con 10.7 % de los pesos, frente a 90.9 % del denso y 86.3 % desde θ_0. Que el ticket supere al denso en el ViT puede ser un efecto regularizador de la poda; no se probó. Todo con una semilla (ver Limitaciones).

H1, complejidad y rewinding: parcial. La ResNet necesita rewinding; el MLP, la CNN y el ViT de 20 épocas no; el ViT largo solo con poca densidad (+3.4 puntos con 7.5 % de los pesos y +3.3 con 4.9 %). La relación no es monótona con la complejidad, porque el ViT, la más compleja, no lo necesita con densidades medias. Cada arquitectura tiene uno o dos puntos de rewind y difieren en más que su complejidad (datos, optimizadores y *schedules*, alcance y tasa de poda, épocas), así que los datos son compatibles con la hipótesis de forma cualitativa pero no la prueban.

H2, warmup y rewinding: compatible, no probada. La ResNet (sin warmup, *learning rate* 0.1) necesita rewinding y el ViT (con warmup) funciona desde θ_0, lo que concuerda con que el warmup haga el trabajo del rewinding. Pero el warmup nunca se varió dentro de una misma arquitectura, y las dos difieren en optimizador, normalización y datos. La prueba directa sería la ResNet con warmup y el ViT sin él.

H3a, poda global frente a por capa (CNN): apoyada. Con poda por capa el ticket colapsa con 10.7 % de los pesos (53.0 % frente a 61.2 % del control); con poda global no (75.7 % frente a 69.4 %). Las máscaras muestran el mecanismo: con poda global conv1 conserva 88 % de sus pesos y no hay filtros ni neuronas vacíos; con poda por capa hay 7 filtros vacíos. El resultado también afecta la comparación entre arquitecturas: el MLP y la primera CNN podaron por capa y la ResNet y el ViT globalmente, así que parte de lo que parece una diferencia entre arquitecturas puede ser una diferencia entre tipos de poda. La limitación principal es que 02b cambia a la vez el alcance de la poda y cuántos pesos conserva cada capa; el experimento no separa ambas cosas.

Tamaño de los pesos. La poda global quita los pesos más pequeños de toda la red, así que lo que sobrevive en cada capa depende de qué tan grandes son sus pesos. En la CNN, los pesos de conv1 son unas 12 veces más grandes que los de fc1 (mediana de 0.105 frente a 0.0086): conv1 tiene solo 1,728 pesos y fc1 más de 2 millones. Un umbral global tira primero los de fc1 y casi no toca conv1; la poda por capa obliga a conv1 a perder lo mismo que fc1, y eso deja filtros y neuronas vacíos. En el ViT pasa algo parecido pero menos claro: `head` y `patch_embed` tienen los pesos más grandes y sobreviven más.

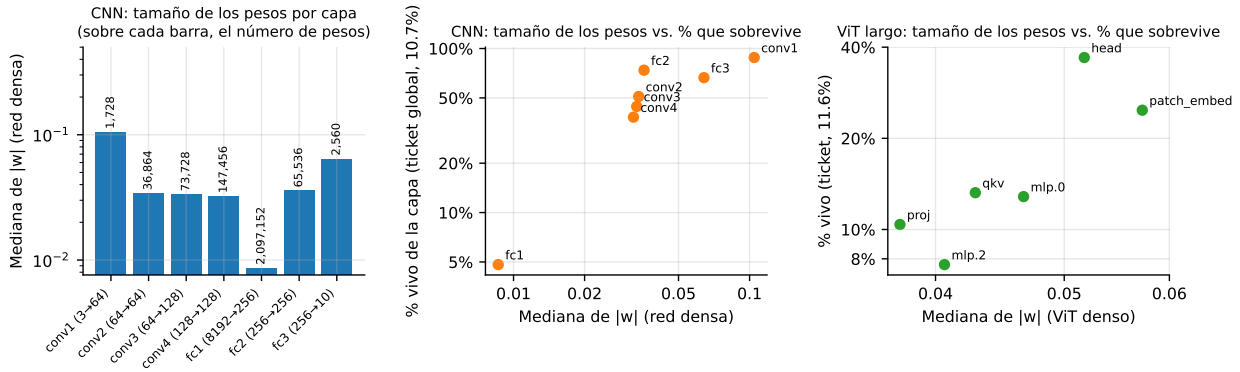


Figura 16: Tamaño de los pesos de cada capa en la red densa y fracción que sobrevive a la poda global. Izquierda: cada barra es una capa de la CNN (sobre la barra, su número de pesos). Centro y derecha: cada punto es una capa de la CNN (10.7 % de los pesos) o un tipo de capa del ViT (11.6 %); más a la derecha = pesos más grandes, más arriba = más pesos vivos. Una sola corrida: es una asociación, no una prueba.

H3b, redes grandes necesitan poda global: consistente, no probada. Con la CNN se puede concluir que la poda por capa causó el colapso y que el mecanismo es el cuello de botella en las capas pequeñas: tienen los pesos más grandes y se les obligó a perder la misma fracción que a la capa enorme y redundante. Eso *no* prueba que toda red grande necesite poda global: ResNet y ViT no se podaron por capa; el MLP sí, sin problemas (97.2 % con 3.5 % de los pesos, aunque su capa de salida tiene 1,000 pesos); y la CNN, con una capa que concentra el 86 % de los pesos, es el caso más favorable para la poda global. Es razonable esperar que el problema crezca con la heterogeneidad de tamaños (el ViT va de 1,920 pesos en `head` a 442,368 en `mlp`, y [1] usa poda global en las redes profundas), pero la prueba directa sería repetir el ViT (unos 45 min de GPU) y la ResNet (1 h 45 min) con poda por capa.

Inspección del ViT. La poda global no es pareja: `head` y `patch_embed` conservan más pesos y `mlp.2` es la más podada; dentro de los bloques, `mlp.0` pierde más en los tardíos y `proj` al revés. No se probó por qué. Por clase, el ticket queda dentro del ruido y el control aleatorio pierde en todas. Como la atención no predice la oclusión, lo que se puede decir del ViT sobre mapas de atención es descriptivo (Sección 6.6): los del control son distintos, pero no se sabe por qué.

Otras vistas revisadas. Durante la inspección se generaron y se revisaron bastantes más gráficas de las que aparecen en este informe: atención por cabeza, entropía de la atención, gradientes en el paso 0, IoU entre las máscaras del ticket y del rewinding, forma de las máscaras y fracción viva por posición del kernel, si los pesos que sobreviven ya eran grandes en la inicialización, matrices de confusión y una imagen por clase. No se incluyeron porque repiten lo que ya muestran otras figuras o porque no sostienen una conclusión por sí solas. Todas están en `notebooks/07_inspection.py` (y en `rendered/07_inspection.html`).

7.2. Limitaciones

Incertidumbre. Una sola semilla por experimento y 10,000 imágenes de prueba. El intervalo binomial del 95 % sobre una accuracy de ∼88 % es de ±0.6 puntos; las diferencias entre ticket y

control (5–9 puntos), entre ticket y denso a 27.5 % de los pesos (+3 puntos) y el colapso de la CNN (−8 puntos) lo superan; ticket frente a denso con 7.5 % de los pesos no. Este intervalo mide solo el ruido del conjunto de prueba, no la variación entre semillas, que no se midió y puede ser mayor (por ejemplo, el denso de la CNN dio 73.0 % y 74.0 % en dos corridas).

Otras limitaciones.

- Una sola semilla por experimento, sin barras de error ni pruebas de significancia.
- No se barrió el punto de rewind k; no se puede decir *cuándo* se fija el ticket en cada arquitectura.
- El warmup no se manipuló (H2).
- Presupuesto de cómputo: el ViT largo tarda ∼5 horas por corrida; un barrido de k sobre cuatro arquitecturas con varias semillas queda fuera del alcance.
- Desviaciones respecto a las fuentes (épocas por ronda, tasas de poda, *learning rate* del ViT sin ajustar): Apéndice A.
- No se probó si un ticket encontrado en un dataset sirve en otro.

8. Conclusión

Se reprodujo la LTH en cuatro arquitecturas y se probaron dos decisiones que cambian el resultado: a qué punto hacer rewind y cómo podar. Con una semilla por experimento, lo que se puede afirmar es lo siguiente.

1. **Los tickets existen en las cuatro arquitecturas.** El ticket iguala o supera a la red densa hasta 3.5 % de los pesos en el MLP, 10.7 % en la CNN (con poda global) y ∼12 % en el ViT; en la ResNet solo con rewinding a la época 1 (89.6 % con 10.7 %, mientras que desde θ_0 no supera al control aleatorio: 86.3 % frente a 86.5 %). Como el control queda por debajo del ticket (salvo en esos casos), importan los valores iniciales y no solo la estructura de la máscara.
2. **El rewind hace falta según la arquitectura y el entrenamiento.** La ResNet lo necesita; el MLP, la CNN y el ViT de 20 épocas no; el ViT largo solo por debajo de ∼8 % de los pesos (+3.4 y +3.3 puntos). El patrón es compatible con que el warmup cumpla el papel del rewinding, pero el warmup nunca se varió, así que no se puede separar de las otras diferencias entre arquitecturas.
3. **La forma de podar cambia el resultado.** En la CNN, con poda por capa el ticket colapsa con 10.7 % de los pesos (53.0 % frente a 61.2 % del control) y deja 7 de los 64 filtros de la primera convolución vacíos; con poda global no (75.7 % frente a 69.4 %). Las capas pequeñas tienen los pesos más grandes, y la poda por capa les exige perder lo mismo que a la capa enorme. Que las redes más grandes necesiten poda global es consistente con esto, pero solo se comparó en la CNN.
4. **El ViT, el caso más estudiado.** El primer baseline (20 épocas) estaba subentrenado; con 100 épocas la red densa llega a 86.0 % y el ticket la supera hasta ∼12 % de los pesos. La poda es desigual entre capas (`head` y `patch_embed` conservan más, `mlp.2` es la más podada). La atención del `[CLS]` no predice qué parches importan (correlación ≈ 0 con la oclusión), así que se usa solo como descripción.

Alcance. Una semilla por experimento y ∼10,000 imágenes de prueba (±0.6 puntos de ruido): diferencias de ∼1 punto no son resultados. Lo más sólido es la existencia de tickets (H0) y el efecto del alcance de la poda en la CNN (H3a), cuyas diferencias superan el ruido; lo más débil es la relación entre warmup y rewinding (H2), que solo se observó.

8.1. Trabajo futuro

- Probar H2 de forma directa: ResNet con warmup y ViT sin warmup.
- Barrer el punto de rewind k en las cuatro arquitecturas, con varias semillas y pruebas de significancia (McNemar sobre las predicciones guardadas es la más barata).
- Probar H3b: repetir ViT y ResNet con poda por capa; y separar el efecto del alcance de la poda de la cantidad de pesos por capa (poda por capa con un mínimo por capa).
- Control aleatorio con valores permutados; transferencia de tickets entre datasets (SVHN).

Referencias

[1] Jonathan Frankle y Michael Carbin. «The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks». En: *International Conference on Learning Representations (ICLR)*. 2019. URL: https://arxiv.org/abs/1803.03635.

[2] Jonathan Frankle et al. *Stabilizing the Lottery Ticket Hypothesis*. arXiv preprint. 2019. URL: https://arxiv.org/abs/1903.01611.

[3] Bowen Shen et al. «Data Level Lottery Ticket Hypothesis for Vision Transformers». En: *International Joint Conference on Artificial Intelligence (IJCAI)*. 2023. URL: https://arxiv.org/abs/2211.01484.

[4] Alex Renda, Jonathan Frankle y Michael Carbin. «Comparing Rewinding and Fine-tuning in Neural Network Pruning». En: *International Conference on Learning Representations (ICLR)*. 2020. URL: https://arxiv.org/abs/2003.02389.

[5] Hugo Touvron et al. «Training data-efficient image transformers & distillation through attention». En: *International Conference on Machine Learning (ICML)*. 2021. URL: https://arxiv.org/abs/2012.12877.

A. Desviaciones respecto a las fuentes

Elemento	Fuente	Este proyecto
Épocas por ronda, MLP	50K iteraciones [1]	5 épocas (∼5K iteraciones)
Tasa de poda, MLP	10 % en la capa de salida	20 % en todas
Tasas de poda, CNN	conv 10 % / lineales 20 %	20 % en todas
Batch, CNN	60	64 (30 épocas)
Validación, ResNet	45K train / 5K val	50K train, sin validación
Punto de rewind, ResNet	iteración 500 [2]	época 1 = iteración 391
Normalización, ResNet	resta de media por píxel	media y desviación por canal
Estado del optimizador al hacer rewind	continuo	se reinicia en la época de rewind
Tasa de poda, ViT	20 %	35 % (menos rondas para llegar a ∼3 %)
Learning rate, ViT	$5 \cdot 10^{-4} \times$ batch/512 $= 1.25 \cdot 10^{-4}$ [5]	10^{-3} (sin ajustar)
Warmup, ViT	5 épocas	2 de 20 (04) y 5 de 100 (05)
Semillas	promedio de varias corridas	una sola

Tabla 9: Desviaciones respecto a las fuentes (fuentes en la bibliografía).

B. Cómo reproducir

```
cd notebooks && uv run python 02b_cnn_global.py   # un experimento
uv run python report/make_figures.py              # figuras y tablas del informe
cd report && latexmk -pdf informe.tex             # este documento
cd notebooks && uv run marimo run 08_demo.py      # demo interactiva
```